

Multiple Programming Paradigms — Multi-Paradigm Use — Answers

Q1. Write a Java program using both class and lambda (OOP + functional).

The following program defines a class (object-oriented style) that accepts a functional interface implemented using a lambda expression (functional style), combining both paradigms in a single program.

```
import java.util.function.Function;

class Calculator {
    private String name;

    Calculator(String name) {
        this.name = name;
    }

    int applyOperation(int value, Function<Integer, Integer> operation) {
        System.out.println("Using calculator: " + name);
        return operation.apply(value);
    }
}

public class MultiParadigmDemo {
    public static void main(String[] args) {
        Calculator calc = new Calculator("MyCalc"); // OOP: object creation

        Function<Integer, Integer> square = x -> x * x; // Functional: lambda

        int result = calc.applyOperation(5, square);
        System.out.println("Result: " + result);
    }
}
```

Here, Calculator is a regular class with encapsulated state, representing the object-oriented part. The lambda expression `x -> x * x` represents a piece of behavior passed as a value into `applyOperation`, representing the functional part. The program prints Result: 25.

Q2. Why is Java considered a multi-paradigm language?

Java is considered a multi-paradigm language because it supports and combines features from more than one style of programming within a single language, rather than restricting the programmer to only one approach:

- Object-oriented paradigm: Java's core design centers on classes, objects, inheritance, encapsulation, and polymorphism.
- Procedural paradigm: static methods and straightforward, step-by-step logic allow code to be written in a procedural style, especially within a single class or main method.
- Functional paradigm: lambda expressions, functional interfaces, and the Stream API (added in Java 8) allow functions to be treated as values and support declarative, functional-style data processing.
- Generic programming: Java generics allow classes and methods to operate on different data types while maintaining type safety.

Because a single Java program can freely mix object-oriented structure, procedural logic, and functional-style expressions, depending on which is most appropriate for a given task, Java is classified as a multi-paradigm language rather than a purely object-oriented one.

Q3. Convert a procedural Java code into an object-oriented version.

Consider the following procedural code that manages a simple student's marks using standalone variables and static methods.

Procedural version:

```
public class StudentProcedural {
    static String name = "Ravi";
    static int marks = 85;

    static void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Marks: " + marks);
    }

    static String getGrade() {
        if (marks >= 80) return "A";
        else if (marks >= 60) return "B";
        else return "C";
    }

    public static void main(String[] args) {
        displayDetails();
        System.out.println("Grade: " + getGrade());
    }
}
```

```
}
```

Object-oriented version:

```
class Student {  
    private String name;  
    private int marks;  
  
    Student(String name, int marks) {  
        this.name = name;  
        this.marks = marks;  
    }  
  
    void displayDetails() {  
        System.out.println("Name: " + name);  
        System.out.println("Marks: " + marks);  
    }  
  
    String getGrade() {  
        if (marks >= 80) return "A";  
        else if (marks >= 60) return "B";  
        else return "C";  
    }  
}  
  
public class StudentOop {  
    public static void main(String[] args) {  
        Student s1 = new Student("Ravi", 85);  
        s1.displayDetails();  
        System.out.println("Grade: " + s1.getGrade());  
    }  
}
```

In the object-oriented version, name and marks become private instance fields bundled inside a Student object, and displayDetails and getGrade become instance methods that operate on that object's own data. This also makes it possible to create multiple independent Student objects, each with its own name and marks, which the static-variable-based procedural version could not easily support.